React Tic-Tac-Toe Tutorial: Progress and Issues Report

Progress Summary: I successfully completed the entire base tutorial. Additionally, I completed all 5 of the advanced Extra Credit improvements suggested at the end of the tutorial, which required modifying the component state and dynamically rendering elements. My final application includes:

1. Text-only display for the current move in the history list.
2. A dynamically generated game board using nested loops (rather than hardcoded squares).
3. A toggle button to sort the move history in ascending or descending order.
4. Visual highlighting for the three squares that cause a win (and a draw state when the board is full).
5. Move location tracking displayed in a (row, col) format in the history list.

Issues Encountered and Resolutions:

1. Local Environment Setup & Terminal Navigation Errors

- **The Issue:** The official tutorial defaults to using CodeSandbox, which hides the build configuration. Because I set this up locally on my machine using Vite, I ran into an `npm error Missing script: "dev"` when trying to start the local server.
- **The Resolution:** The error occurred because I was attempting to run the command from the parent `react`directory rather than the newly generated `tic-tac-toe` project folder. I resolved this by using the `cd` command to navigate into the correct directory where the `package.json` file was located before running the server.

2. Legacy Package Vulnerabilities (`npm audit`)

- **The Issue:** When installing packages locally, `npm` flagged several high and critical vulnerabilities (specifically with `minimist` and `node-static`) and stated there was "No fix available" when running `npm audit fix`.
- **The Resolution:** Because modern Vite installations sometimes rely on older sub-dependencies for local development servers, these flags appeared. I learned that because this is a local learning environment and these are dev-dependencies (not production code being deployed to a live server), these vulnerabilities do not pose a risk to the application and could be safely bypassed.

3. CSS Boilerplate Conflicts

- **The Issue:** Vite automatically generates boilerplate styling in files like `App.css` and `index.css`. When I first applied the tutorial's React code, the board did not render as a grid; instead, it rendered as tiny, unstyled circles.
- **The Resolution:** The default Vite styles were overriding the necessary board layout. I resolved this by completely deleting the contents of the Vite-generated `App.css` and `index.css` files, and pasting the vanilla CSS provided by the React tutorial exclusively into `index.css`.

4. Configuration Tooling (Linter Selection)

- **The Issue:** During the Vite setup process, the CLI prompted me to choose a linter (ESLint vs. Oxlint). The React tutorial does not cover this step because it assumes a pre-configured environment.
- **The Resolution:** I researched the options and selected ESLint. While Oxlint is newer and faster, ESLint remains the industry standard, making it the most appropriate choice for an academic full-stack project.